NAME: A.JOTHEESH
REG NO:192511029

Experiment 1: Lexical Analyzer

Aim

To write a C program to implement a lexical analyzer that identifies identifiers, constants, and operators and ignores spaces, tabs, newlines, and comments.

Code

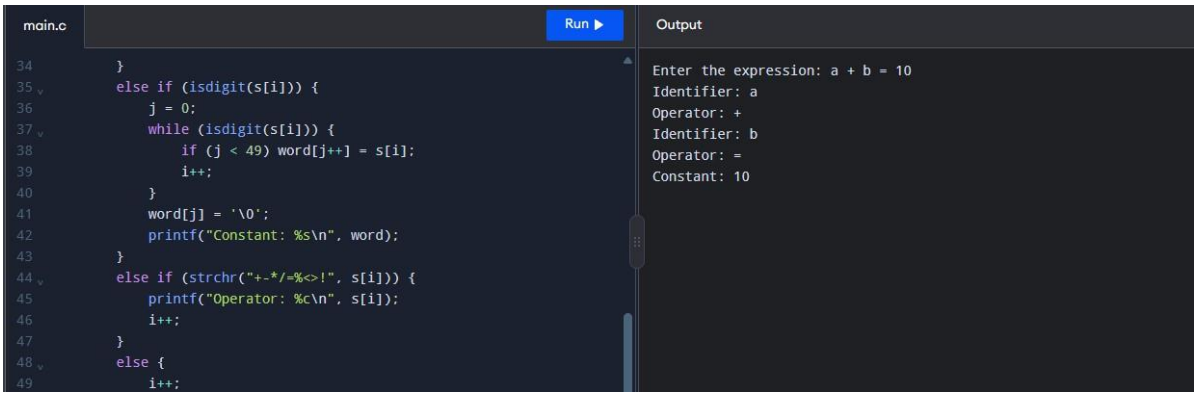
```
#include  <stdio.h>
#include <ctype.h>
#include <string.h>
int main() {
   char s[500], word[50];
   int i = 0, j;
   printf("Enter the expression: ");
   fgets(s, sizeof(s), stdin);

   while (s[i] != '\0') {
      if (isspace(s[i])) {
         i++;
      }
      else if (s[i] == '/' && s[i+1] == '/') {
         break;
      }
      else if (s[i] == '/' && s[i+1] == '*') {
         i += 2;
         while (s[i] && !(s[i] == '*' && s[i+1] == '/'))
            i++;
         if (s[i]) i += 2;
      }
      else if (isalpha(s[i]) || s[i] == '_') {
         j = 0;
         while (isalnum(s[i]) || s[i] == '_') {
            if (j < 49) word[j++] = s[i];
            i++;
         }
         word[j] = '\0';
         printf("Identifier: %s\n", word);
      }
      else if (isdigit(s[i])) {
         j = 0;
         while (isdigit(s[i])) {
            if (j < 49) word[j++] = s[i];
            i++;
         }
```

```
word[j] = '\0';
```

```
            printf("Constant: %s\n", word);
        }
        else if (strchr("+-*/=%<>!", s[i])) {
            printf("Operator: %c\n", s[i]);
            i++;
        }
        else {
            i++;
        }
    }
    return 0;
}
```

Output:



Experiment 2: Lexical Analyzer to Identify Comments

1. Aim

To write a C program to identify whether the given input is a single-line comment (//) or a multi-line comment (/* */).

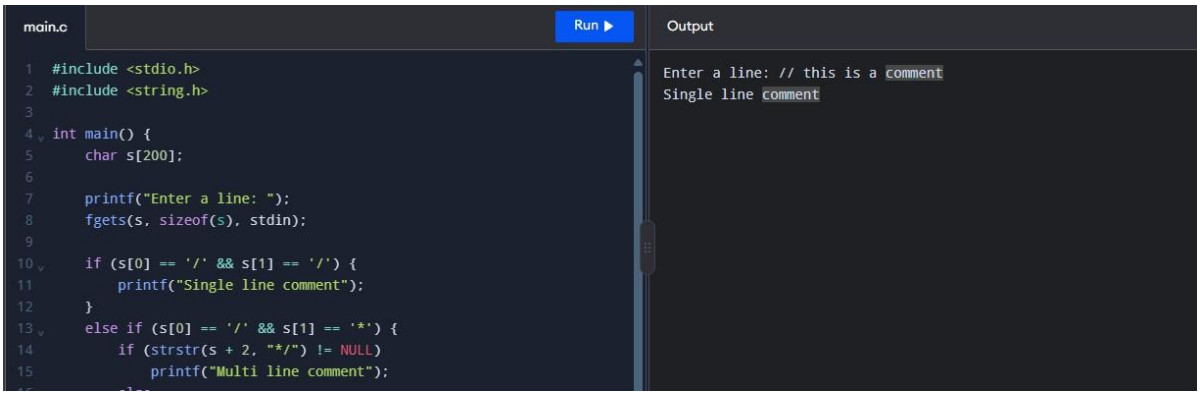
2. Code (C Program)

```
#include <stdio.h>
#include <string.h>

int main() {
   char s[200];
   printf("Enter a line: ");
   fgets(s, sizeof(s), stdin);
   if (s[0] == '/' && s[1] == '/') {
      printf("Single line comment");
   }
   else if (s[0] == '/' && s[1] == '*') {
      if (strstr(s + 2, "*/") != NULL)
         printf("Multi line comment");
      else
         printf("Incomplete comment");
   }
   else {
      printf("Not a comment");
   }

   return 0;
}
```

Output



Experiment 3: Lexical Analyzer

1. Aim

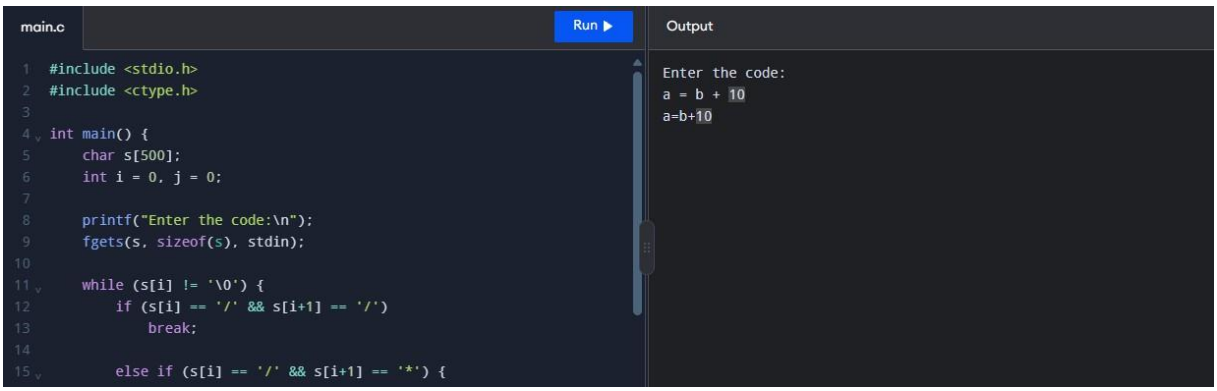
To write a C program to implement a lexical analyzer that ignores redundant spaces, tabs, newlines, and comments.

2. Code (C Program)

```
#include <stdio.h>
#include <ctype.h>

int main() {
   char s[500];
   int i = 0, j = 0;
   printf("Enter the code:\n");
   fgets(s, sizeof(s), stdin);
   while (s[i] != '\0') {
      if (s[i] == '/' && s[i+1] == '/')
         break;
      else if (s[i] == '/' && s[i+1] == '*') {
         i += 2;
         while (s[i] && !(s[i] == '*' && s[i+1] == '/'))
            i++;
         if (s[i]) i += 2;
      }
      else if (isspace(s[i])) {
         i++;
      }
      else {
         printf("%c", s[i]);
         i++;
      }
   }
   return 0;
}
```

Output



Experiment 4: Lexical Analyzer to Validate Arithmetic Operators

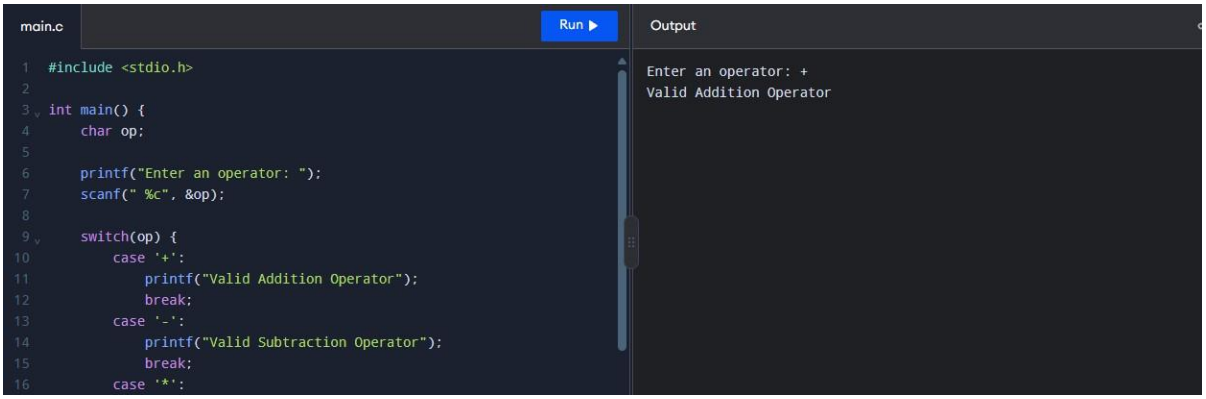
1. Aim

To write a C program to identify and validate arithmetic operators (+, -, *, /) using a lexical analyzer.

2. Code (C Program)

```
#include <stdio.h>
int main() {
   char op;
   printf("Enter an operator: ");
   scanf(" %c", &op);
   switch(op) {
      case '+':
         printf("Valid Addition Operator");
         break;
      case '-':
         printf("Valid Subtraction Operator");
         break;
      case '*':
         printf("Valid Multiplication Operator");
         break;
      case '/':
         printf("Valid Division Operator");
         break;
      default:
         printf("Invalid Operator");
   }
```

Output



Experiment 5: Lexical Analyzer to Count Whitespaces and Newlines

1. Aim

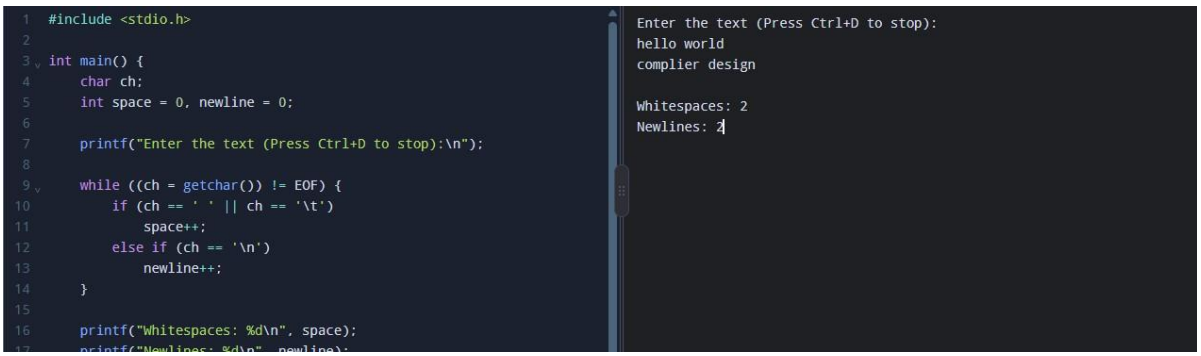
To write a C program to count the number of whitespaces and newline characters in a given input.

2. Code (C Program)

```
#include <stdio.h>

int main() {
    char ch;
    int space = 0, newline = 0;
    printf("Enter the text (Press Ctrl+D to stop):\n");
    while ((ch = getchar()) != EOF) {
        if (ch == ' ' || ch == '\t')
            space++;
        else if (ch == '\n')
            newline++;
    }
    printf("Whitespaces: %d\n", space);
    printf("Newlines: %d\n", newline);
    return 0;
}
```

Output:



Experiment 6: Lexical Analyzer to Validate Identifier

1. Aim

To write a C program to check whether a given identifier is valid or invalid.

2. Code (C Program)

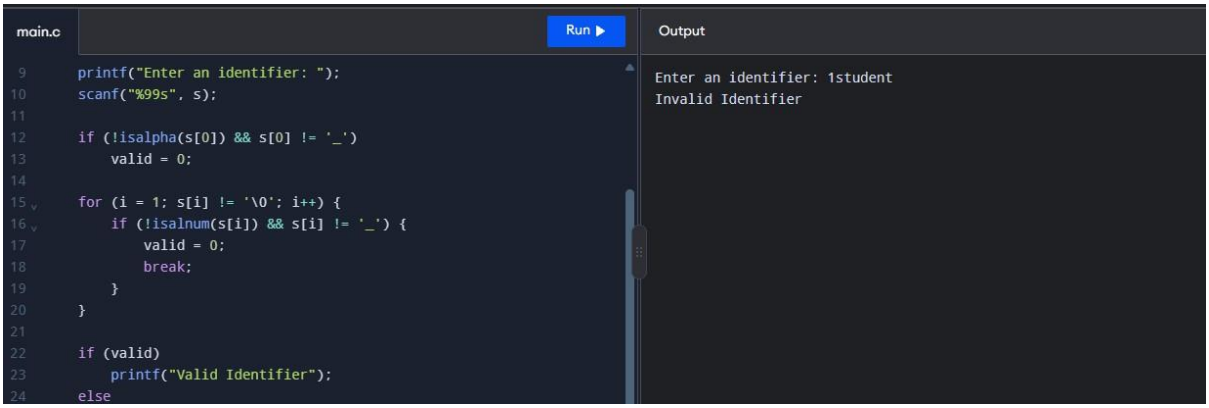
```
#include  <stdio.h>
#include  <ctype.h>
#include <string.h>
int main() {
    char s[100];
```

```
    int i, valid = 1;
    printf("Enter an identifier: ");
    scanf("%99s", s);
    if (!isalpha(s[0]) && s[0] != '_')
        valid = 0;
    for (i = 1; s[i] != '\0'; i++) {
        if (!isalnum(s[i]) && s[i] != '_') {
            valid = 0;
            break;
        }
    }
    if (valid)
        printf("Valid Identifier");
    else
        printf("Invalid Identifier");

    return 0;
}
```

Output:





Experiment 7: FIRST() – Predictive Parser

1. Aim

To write a C program to find the FIRST() sets for the given grammar.

2. Code:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

char grammar[20][50], first[26][50];
int n, visiting[26];

void add(int x, char c) {
    if (strchr(first[x], c) == NULL) {
        int len = strlen(first[x]);
        first[x][len] = c;
        first[x][len + 1] = '\0';
    }
```

```
}

void findFirst(char c) {
   int i, j, k, x = c - 'A';

   if (visiting[x]) return;
   visiting[x] = 1;

   for (i = 0; i < n; i++) {
      if (grammar[i][0] != c) continue;

      char *rhs = strchr(grammar[i], '=');
      if (rhs == NULL) continue;
      rhs++;
            if (ch == '#') {
               j++;
               break;
            }
            else if (!isupper((unsigned char)ch)) {
               add(x, ch);
               nullable = 0;
               j++;
               break;
            }
            else {
               findFirst(ch);
               int y = ch - 'A';

               for (k = 0; first[y][k]; k++)
                  if (first[y][k] != '#')
                     add(x, first[y][k]);

               if (strchr(first[y], '#') == NULL) {
                  nullable = 0;
                  j++;
                  break;
               }
            }
            j++;
         }

         if (nullable) add(x, '#');

         while (rhs[j] && rhs[j] != '|') j++;
         if (rhs[j] == '|') j++;
      }
   }
   visiting[x] = 0;
}
int main() {
```

```
    int i;
    for (i = 0; i < n; i++)
        findFirst(grammar[i][0]);
    printf("\nFIRST Sets:\n");
    for (i = 0; i < n; i++) {
        int x = grammar[i][0] - 'A';
        printf("FIRST(%c) = {%s}\n",
               grammar[i][0], first[x]);
    }

    return 0;
}
```

Output:



Experiment 8: FOLLOW() – Predictive Parser

1. Aim

To write a C program to find the FOLLOW() sets for the given grammar.

2. Code (C Program)

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
char grammar[10][50], follow[26][50];
int n;
void add(int x, char c) {
    if (strchr(follow[x], c) == NULL) {
        int len = strlen(follow[x]);
```

```
        follow[x][len] = c;
        follow[x][len + 1] = '\0';
    }
}
int main() {    int i, j, k, change;
    printf("Enter number of productions: ");
    scanf("%d", &n);
    printf("Enter productions:\n");
    for (i = 0; i < n; i++)
        scanf("%49s", grammar[i]);

    add(grammar[0][0] - 'A', '$');
    do {
        char old[26][50];
        memcpy(old, follow, sizeof(follow));
        for (i = 0; i < n; i++) {
            char lhs = grammar[i][0];
            char *rhs = strchr(grammar[i], '=');

            if (rhs == NULL) continue;
            rhs++;

            for (j = 0; rhs[j]; j++) {
                if (!isupper((unsigned char)rhs[j]))
                    continue;
                int x = rhs[j] - 'A';
                int nullable = 1;

                for (k = j + 1; rhs[k]; k++) {
                    if (rhs[k] == '|') break;
                    if (islower((unsigned char)rhs[k])) {
                        add(x, rhs[k]);
                        nullable = 0;
                        break;
                    }

                    if (isupper((unsigned char)rhs[k])) {
                        int m, epsilon = 0;

                        for (m = 0; m < n; m++) {
                            if (grammar[m][0] == rhs[k] &&
                                strstr(grammar[m], "=#"))
                                epsilon = 1;
                        }

                        if (!epsilon) {
                            nullable = 0;
                            break;
                        }
                    }
                }
```

```
            if (nullable) {
                char *p = follow[lhs - 'A'];
                while (*p) add(x, *p++);
            }
        }
    }
    change = memcmp(old, follow, sizeof(follow));
} while (change);

printf("\nFOLLOW Sets:\n");
for (i = 0; i < n; i++) {
    int x = grammar[i][0] - 'A';
    printf("FOLLOW(%c) = {%s}\n",
           grammar[i][0], follow[x]);
}
return 0;
}
```

Output:



Experiment 9: Elimination of Left Recursion

1. Aim

To write a C program to eliminate left recursion from the given context-free grammar (CFG).

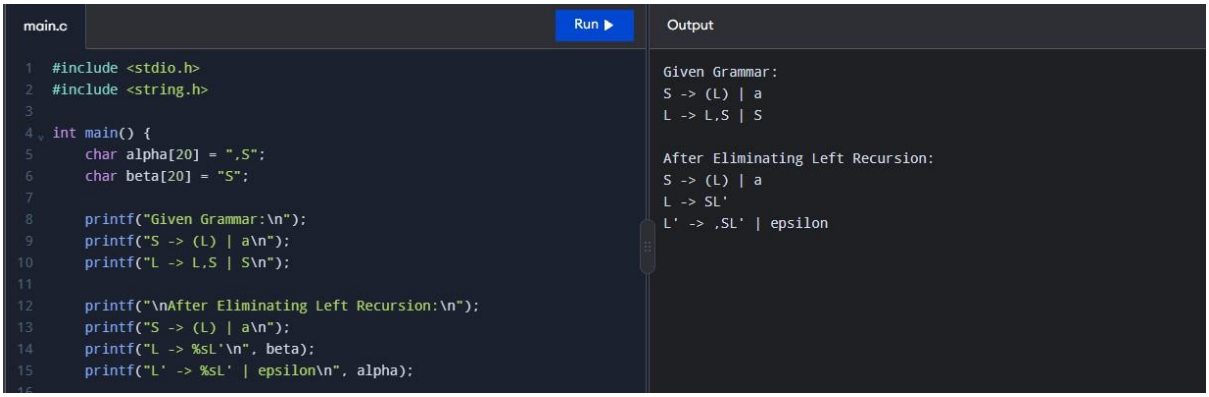
2. Code (C Program)

```
#include <stdio.h>
#include <string.h>
int main() {
   char nt, p1[50], p2[50];
   int i;
   printf("Enter non-terminal: ");
   scanf(" %c", &nt);
   printf("Enter first production: ");
   scanf("%49s", p1);

 printf("Enter second production: ");
   scanf("%49s", p2);
   if (p1[0] == nt) {
      printf("\nAfter Eliminating Left Recursion:\n");
      printf("%c -> %s%c'\n", nt, p2, nt);
            nt, p1 + 1, nt);
   }
   else if (p2[0] == nt) {
      printf("\nAfter Eliminating Left Recursion:\n");
      printf("%c -> %s%c'\n", nt, p1, nt)
            nt, p2 + 1, nt);
```

```
    }
    else {
        printf("No Left Recursion");
    }
    return 0;
}
```

Output:



Experiment 10: Elimination of Left Factoring

1. Aim

To write a C program to eliminate left factoring from the given context-free grammar (CFG).

2. Code (C Program)

```
#include <stdio.h>
#include <string.h>

int main() {
    char p1[50], p2[50], p3[50];
    int i = 0;

    printf("Enter first production: ");
    scanf("%49s", p1);

    printf("Enter second production: ");
    scanf("%49s", p2);

    printf("Enter third production: ");
    scanf("%49s", p3);

    while (p1[i] && p2[i] && p1[i] == p2[i])
```

```
        i++;

    if (i == 0) {
        printf("No left factoring needed\n");
    }
    else {
        printf("\nAfter Left Factoring:\n");
        printf("S -> %.*sS' | %s\n", i, p1, p3);
        printf("S' -> %s | %s\n",
               p1[i] ? p1 + i : "epsilon",
               p2[i] ? p2 + i : "epsilon");
    }

    return 0;
}
```

Output:

